

Отчёт по лабораторной работе №13

**Программирование в командном процессоре ОС UNIX. Ветвления
и циклы**

Юсупова Амина Руслановна

Содержание

1	Цель работы	5
2	Теоретические сведения	6
2.1	Команда <code>getopts</code>	6
2.2	Задание 1	7
2.3	Задание 2	8
2.4	Задание 3	11
2.5	Задание 4	12
3	Выводы	15
4	Ответы на контрольные вопросы	16
5	Ответы на контрольные вопросы	17
5.1	1. Каково предназначение команды <code>getopts</code> ?	17
5.2	2. Какое отношение метасимволы имеют к генерации имён файлов?	18
5.3	5. Для чего нужны команды <code>false</code> и <code>true</code> ?	20
5.4	6. Что означает строка <code>if test -f mans/i.\$s</code> , встреченная в командном файле?	22
5.5	7. Объясните различия между конструкциями <code>while</code> и <code>until</code>	23

Список иллюстраций

2.1	Скрипт	7
2.2	Исполняемый файл	7
2.3	Запуск	8
2.4	Скрипт	9
2.5	Скрипт	9
2.6	Запуск	10
2.7	Скрипт	11
2.8	Исполняемый файл	11
2.9	Запуск	12
2.10	тестовые файлы	13
2.11	Скрипт	13
2.12	Запуск	14

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Теоретические сведения

2.1 Команда getopts

Команда `getopts` используется в `bash` для разбора аргументов командной строки. Она позволяет обрабатывать опции (ключи) и их аргументы, что делает скрипты более гибкими и удобными для пользователя. Синтаксис:

```
while getopts "опции" переменная; do
    case $переменная in опция) действия ;;
    esac
done
```

Управляющие конструкции `bash`

Конструкция	Назначение
if-then-else	Условное выполнение команд
for	Цикл с перебором списка значений
while	Цикл с предусловием
until	Цикл с постусловием
case	Множественное ветвление

Команды `break` и `continue` - `break` — прерывает выполнение цикла

- `continue` — прерывает текущую итерацию и переходит к следующей

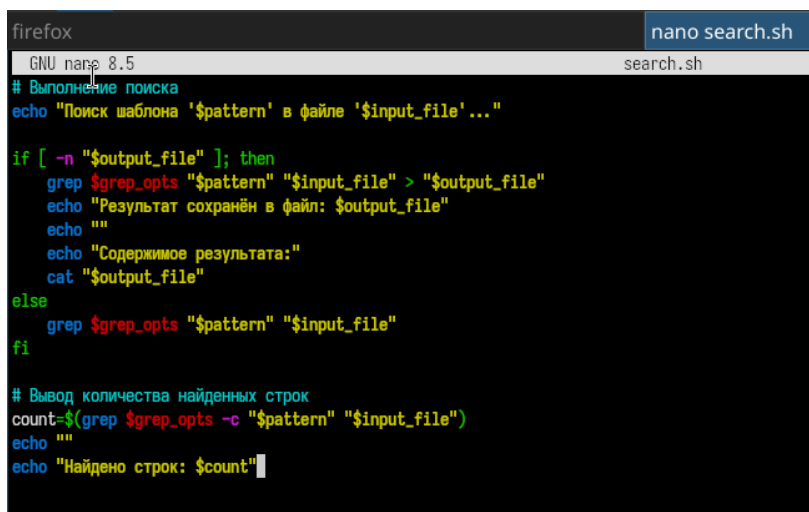
Команды true и false - true — всегда возвращает код завершения 0 (истина)

- false — всегда возвращает код завершения 1 (ложь) # Выполнение лабораторной работы

2.2 Задание 1

2.2.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `search.sh`. А далее я написала скрипт, который при запуске будет искать в указанном файле нужные строки, определяемые ключом `-r`



```
firefox nano search.sh
GNU nano 8.5 search.sh
# Выполнение поиска
echo "Поиск шаблона '$pattern' в файле '$input_file'..."

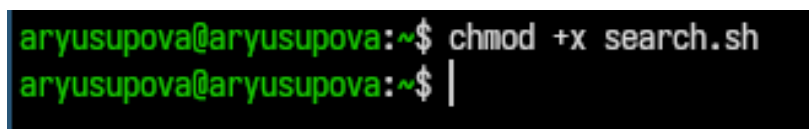
if [ -n "$output_file" ]; then
    grep $grep_opts "$pattern" "$input_file" > "$output_file"
    echo "Результат сохранён в файл: $output_file"
    echo ""
    echo "Содержимое результата:"
    cat "$output_file"
else
    grep $grep_opts "$pattern" "$input_file"
fi

# Вывод количества найденных строк
count=$(grep $grep_opts -c "$pattern" "$input_file")
echo ""
echo "Найдено строк: $count"
```

Рисунок 2.1: Скрипт

2.2.2 2. Исполняемый скрипт

Теперь я делаю скрипт исполняемым, чтобы программа работала



```
argyusupova@argyusupova:~$ chmod +x search.sh
argyusupova@argyusupova:~$
```

Рисунок 2.2: Исполняемый файл

2.2.3 3. Запуск скрипта

Теперь я запускаю скрипт в командной строке

```
argusupova@argusupova:~$ ./search.sh -i test.txt -p hello
Поиск шаблона 'hello' в файле 'test.txt'...
Hello world
hello friends
HELLO everyone
hello again
Hello World

Найдено строк: 5
argusupova@argusupova:~$ ./search.sh -i test.txt -p Hello -C -n
Поиск шаблона 'Hello' в файле 'test.txt'...
1:Hello world
6:Hello World

Найдено строк: 2
argusupova@argusupova:~$ ./search.sh -i test.txt -p hello -o result.txt
Поиск шаблона 'hello' в файле 'test.txt'...
Результат сохранён в файл: result.txt

Содержимое результата:
Hello world
hello friends
HELLO everyone
hello again
Hello World

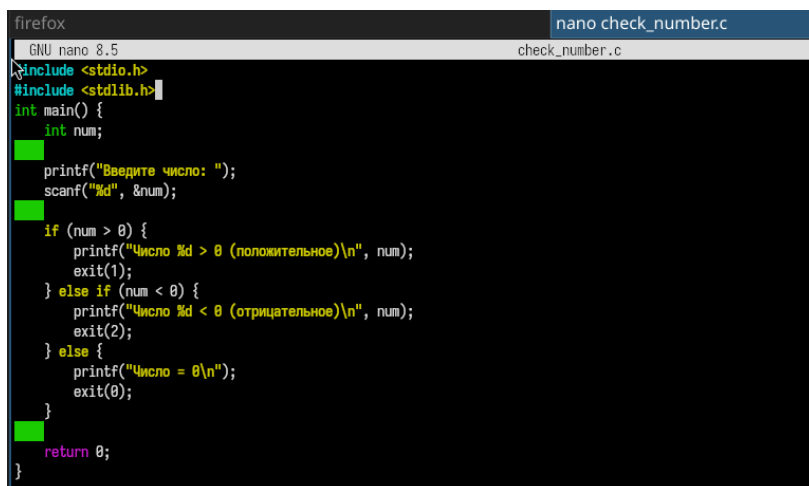
Найдено строк: 5
```

Рисунок 2.3: Запуск

2.3 Задание 2

2.3.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `check_number.c`. А далее я написала скрипт, который вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю.



```
GNU nano 8.5 nano check_number.c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int num;

    printf("Введите число: ");
    scanf("%d", &num);

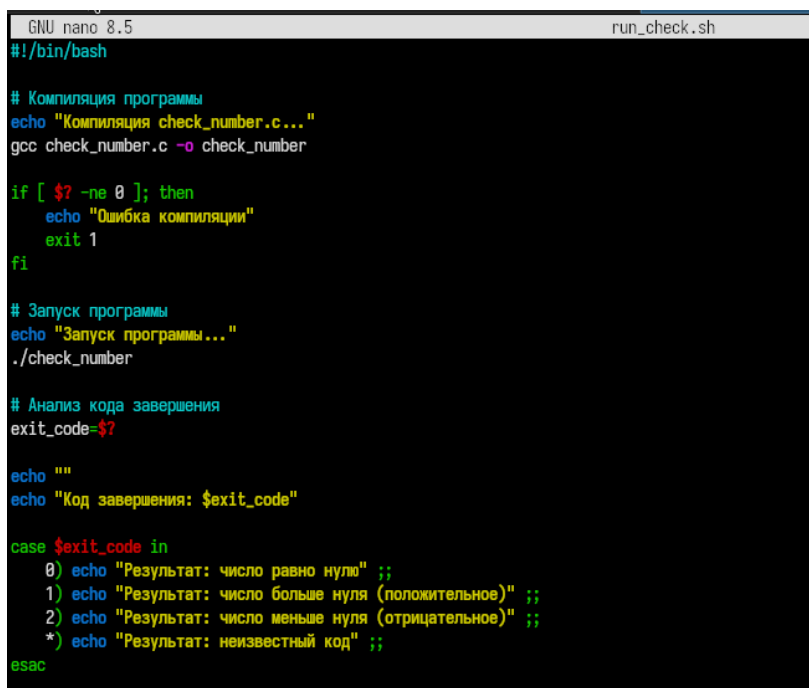
    if (num > 0) {
        printf("Число %d > 0 (положительное)\n", num);
        exit(1);
    } else if (num < 0) {
        printf("Число %d < 0 (отрицательное)\n", num);
        exit(2);
    } else {
        printf("Число = 0\n");
        exit(0);
    }

    return 0;
}
```

Рисунок 2.4: Скрипт

2.3.2 2. Создание командного файл

Создаю командный файл run_check.sh, где я написала скрипт для работы файла check_number.c



```
GNU nano 8.5 run_check.sh
#!/bin/bash

# Компиляция программы
echo "Компиляция check_number.c..."
gcc check_number.c -o check_number

if [ $? -ne 0 ]; then
    echo "Ошибка компиляции"
    exit 1
fi

# Запуск программы
echo "Запуск программы..."
./check_number

# Анализ кода завершения
exit_code=$?

echo ""
echo "Код завершения: $exit_code"

case $exit_code in
    0) echo "Результат: число равно нулю" ;;
    1) echo "Результат: число больше нуля (положительное)" ;;
    2) echo "Результат: число меньше нуля (отрицательное)" ;;
    *) echo "Результат: неизвестный код" ;;
esac
```

Рисунок 2.5: Скрипт

2.3.3 3. Запуск скрипта

Теперь я запускаю скрипт в командной строке

```
argusupova@argusupova:~$ ./run_check.sh
Компиляция check_number.c...
Запуск программы...
Введите число: 5
Число 5 > 0 (положительное)

Код завершения: 1
Результат: число больше нуля (положительное)
argusupova@argusupova:~$ ./run_check.sh
Компиляция check_number.c...
Запуск программы...
Введите число: 2
Число 2 > 0 (положительное)

Код завершения: 1
Результат: число больше нуля (положительное)
argusupova@argusupova:~$ ./run_check.sh
Компиляция check_number.c...
Запуск программы...
Введите число: -1
Число -1 < 0 (отрицательное)

Код завершения: 2
Результат: число меньше нуля (отрицательное)
```

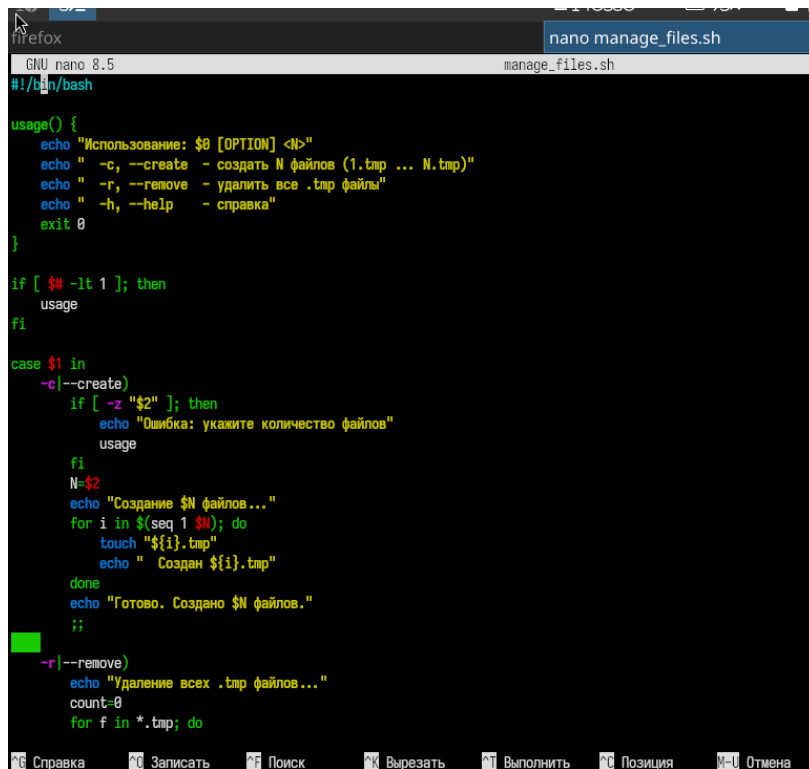
Рисунок 2.6: Запуск

Как видно на картинке программа выводит отрицательное или положительное число вводит пользователь.

2.4 Задание 3

2.4.1 1. Создание рабочего файла и написание кода

Для начала я создала рабочий файл `manage_files.sh`. А далее я написала скрипт, который создает указанное число файлов, пронумерованных последовательно.



```
#!/bin/bash

usage() {
    echo "Использование: $0 [OPTION] <N>"
    echo "  -c, --create  - создать N файлов (1.tmp ... N.tmp)"
    echo "  -r, --remove  - удалить все .tmp файлы"
    echo "  -h, --help   - справка"
    exit 0
}

if [ $# -lt 1 ]; then
    usage
fi

case $1 in
    -c|--create)
        if [ -z "$2" ]; then
            echo "Ошибка: укажите количество файлов"
            usage
        fi
        N=$2
        echo "Создание $N файлов..."
        for i in $(seq 1 $N); do
            touch "${i}.tmp"
            echo "Создан ${i}.tmp"
        done
        echo "Готово. Создано $N файлов."
        ;;
    -r|--remove)
        echo "Удаление всех .tmp файлов..."
        count=0
        for f in *.tmp; do
```

Рисунок 2.7: Скрипт

2.4.2 2. Исполняемый скрипт

Теперь я делаю скрипт исполняемым, чтобы программа работала



```
aryusupova@aryusupova:~$ chmod +x manage_files.sh
aryusupova@aryusupova:~$
```

Рисунок 2.8: Исполняемый файл

2.4.3 3. Запуск скрипта

Теперь я запускаю скрипт в командной строке

```
argusupova@argusupova:~$ ./manage_files.sh -c 5
Создание 5 файлов...
Создан 1.tmp
Создан 2.tmp
Создан 3.tmp
Создан 4.tmp
Создан 5.tmp
Готово. Создано 5 файлов.
argusupova@argusupova:~$ ls -la *.tmp
-rw-r--r--. 1 argusupova argusupova 0 апр 27 00:00 1.tmp
-rw-r--r--. 1 argusupova argusupova 0 апр 27 00:00 2.tmp
-rw-r--r--. 1 argusupova argusupova 0 апр 27 00:00 3.tmp
-rw-r--r--. 1 argusupova argusupova 0 апр 27 00:00 4.tmp
-rw-r--r--. 1 argusupova argusupova 0 апр 27 00:00 5.tmp
argusupova@argusupova:~$ ./manage_files.sh -r
Удаление всех .tmp файлов...
Удалён 1.tmp
Удалён 2.tmp
Удалён 3.tmp
Удалён 4.tmp
Удалён 5.tmp
Готово. Удалено 5 файлов.
```

Рисунок 2.9: Запуск

Как видно на картинке программа создаёт файлы в каталоге, ищет их и удаляет

2.5 Задание 4

2.5.1 1. Создание рабочего файла и написание кода

Для начала я создала тестовые файлы.

```

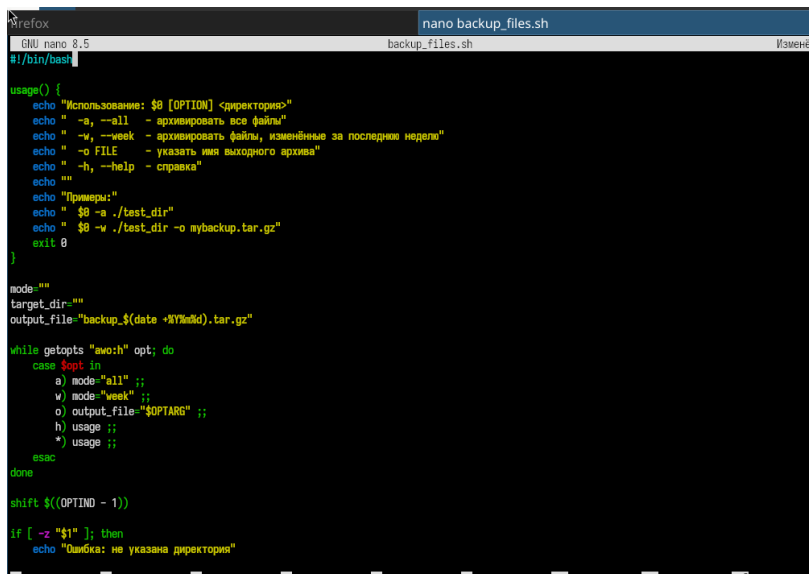
aryusupova@aryusupova:~$ mkdir -p test_dir
aryusupova@aryusupova:~$ cd test_dir
aryusupova@aryusupova:~/test_dir$ touch file1.txt file2.txt file3.txt old_file.txt
aryusupova@aryusupova:~/test_dir$ echo "test content" > new_file.txt
aryusupova@aryusupova:~/test_dir$ cd ..
aryusupova@aryusupova:~$ |

```

Рисунок 2.10: тестовые файлы

2.5.2 2. Создание рабочего файла и написание скрипта

Теперь я создаю рабочий файл и пишу скрипт, который с помощью команды tar запаковывает в архив все файлы в указанной директории.



```

#refox
GNU nano 8.5 backup_files.sh
#!/bin/bash

usage() {
    echo "Использование: $0 [OPTION] <директория>"
    echo "  -a, --all      - архивировать все файлы"
    echo "  -w, --week     - архивировать файлы, измененные за последнюю неделю"
    echo "  -o FILE        - указать имя выходного архива"
    echo "  -h, --help     - справка"
    echo ""
    echo "Примеры:"
    echo "  $0 -a ./test_dir"
    echo "  $0 -w ./test_dir -o mybackup.tar.gz"
    exit 0
}

mode=""
target_dir=""
output_file="backup_$(date +%Y%m%d).tar.gz"

while getopts "awo:h" opt; do
    case $opt in
        a) mode="all" ;;
        w) mode="week" ;;
        o) output_file="$OPTARG" ;;
        h) usage ;;
        *) usage ;;
    esac
done

shift $(($OPTIND - 1))

if [ -z "$1" ]; then
    echo "Ошибка: не указана директория"

```

Рисунок 2.11: Скрипт

2.5.3 3. Запуск скрипта

Теперь я делаю файл исполняемым и запускаю скрипт в командной строке

```
aryusupova@aryusupova:~$ chmod +x backup_files.sh
aryusupova@aryusupova:~$ ./backup_files.sh -a ./test_dir

=====
Архивация директории: ./test_dir
Режим: all
Архив: backup_20260427.tar.gz
=====

Сбор всех файлов...
Найдено файлов: 5
Создание архива...
✓ Архив создан: backup_20260427.tar.gz
Размер: 4,0K
=====

aryusupova@aryusupova:~$ ./backup_files.sh -w ./test_dir -o week_backup.tar.gz

=====
Архивация директории: ./test_dir
Режим: week
Архив: backup_20260427.tar.gz
=====

Сбор файлов, изменённых за последнюю неделю...
Найдено файлов: 5
Создание архива...
✓ Архив создан: backup_20260427.tar.gz
Размер: 4,0K
=====
```

Рисунок 2.12: Запуск

Как видно на картинке командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории, работает корректно.

3 Выводы

В ходе выполнения 13 лабораторной работы я изучила основы программирования в оболочке ОС UNIX/Linux и получила практические навыки написания командных файлов с использованием логических управляющих конструкций, циклов и команды `getopts`.

4 Ответы на контрольные вопросы

5 Ответы на контрольные вопросы

5.1 1. Каково предназначение команды getoptс?

Команда getoptс предназначена для разбора аргументов командной строки в bash-скриптах. Она позволяет обрабатывать опции (ключи) и их аргументы, что делает скрипты более гибкими и удобными для пользователя.

Основные особенности getoptс: - Автоматически обрабатывает стандартные соглашения (опции с дефисом, группировка опций типа -la) - Предоставляет встроенные переменные: - OPTARG — значение аргумента опции (если ожидается) - OPTIND — индекс следующего аргумента для обработки - Обычно используется в цикле while с оператором case

Пример использования:

```
while getoptс "i:o:p:Cn" opt; do
    case $opt in
        i) input_file="$OPTARG" ;;
        o) output_file="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive="C" ;;
        n) line_numbers="n" ;;
        *) usage ;;
    esac
done
```

5.2 2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы используются в `bash` для генерации имён файлов — это называется «подстановка имён файлов» (filename expansion или globbing). Shell автоматически заменяет шаблоны, содержащие метасимволы, на список имён файлов, которые соответствуют этому шаблону.

Основные метасимволы для генерации имён файлов:

Метасимвол	Описание	Пример
<code>*</code>	Любая строка (в том числе пустая)	<code>*.txt</code> — все файлы с расширением <code>.txt</code>
<code>?</code>	Любой один символ file?	<code>.txt</code> — <code>file1.txt</code> , <code>fileA.txt</code>
<code>[abc]</code>	Любой из указанных символов	<code>file[123].txt</code> — <code>file1.txt</code> , <code>file2.txt</code> , <code>file3.txt</code>
<code>[a-z]</code>	Любой символ в указанном диапазоне	<code>[a-z]*</code> — все файлы, начинающиеся с буквы <code>a-z</code>

Пример:

```
echo *.c      # выведет все файлы с расширением .c в текущей директории
ls [0-9]*     # покажет файлы, начинающиеся с цифры
```

5.2.1 3. Какие операторы управления действиями вы знаете?

Оператор	Назначение	Пример
if-then-else	Условное выполнение команд	<pre>if [-f file]; then echo "exists"; else echo "not found"; fi</pre>
for	Цикл с перебором списка значений	<pre>for i in 1 2 3; do echo \$i; done</pre>
while	Цикл с предусловием (выполняется, пока условие истинно)	<pre>while [\$x -lt 5]; do echo \$x; ((x++)); done</pre>
until	Цикл с постусловием (выполняется, пока условие ложно)	<pre>until [\$x -ge 5]; do echo \$x; ((x++)); done</pre>
case	Множественное ветвление (аналог switch)	<pre>case \$var in a) echo "A";; b) echo "B";; *) echo "other";; esac</pre>
break	Прерывание выполнения цикла	break
continue	Переход к следующей итерации цикла	continue
exit	Завершение скрипта с указанным кодом	exit 1

4. Какие операторы используются для прерывания цикла?
5. Для прерывания цикла используются два оператора:

Оператор	Действие
<code>break</code>	Полностью прерывает выполнение цикла. Управление передаётся командам, следующим за <code>done</code>
<code>continue</code>	Прерывает только текущую итерацию цикла и переходит к следующей итерации

Пример:

```
for i in 1 2 3 4 5; do
    if [ $i -eq 3 ]; then
        continue    # пропускаем число 3
    fi
    if [ $i -eq 5 ]; then
        break        # выходим из цикла на числе 5
    fi
    echo $i
done
# Вывод: 1 2 4
```

5.3 5. Для чего нужны команды `false` и `true`?

Команда	Код завершения	Назначение
true	0 (истина)	Всегда возвращает успешный код завершения. Используется для создания бесконечных циклов и в условных конструкциях, где требуется всегда истинное условие
false	1 (ложь)	Всегда возвращает код ошибки. Используется для создания циклов, которые никогда не выполняются, или в комбинации с until

Примеры использования:

```
# Бесконечный цикл (будет работать, пока не прервут)
while true; do
    echo "Работает..."
    sleep 1
done

# Цикл, который не выполнится ни разу
```

```
while false; do
    echo "Это никогда не будет выведено"
done

# Цикл until с false (бесконечный)
until false; do
    echo "Это бесконечный цикл"
done
```

5.4 6. Что означает строка `if test -f mans/i.$s`, встреченная в командном файле?

Данная строка проверяет, существует ли файл с именем, составленным из значений переменных.

Разбор конструкции:

Часть	Описание
<code>test -f</code>	Проверяет, является ли указанный путь обычным файлом
<code>man<i>s</i>/</code>	Директория, где <i>s</i> (, <i>man1</i> /) i.\$s

Что делает:

- Подставляет значение переменной *mans*
- Подставляет значение переменной *\$i* вместо *\$i*
- Подставляет значение переменной *\$s* вместо последнего *\$s*
- Проверяет, существует ли файл по полученному пути

Пример:

```
# Если s="doc", i="1"
# то строка превращается в: if test -f man1/1.doc
# Проверяется существование файла man1/1.doc
```

5.5 7. Объясните различия между конструкциями **while** и **until**.

Характеристика	while	until
Условие выполнения	Выполняется, пока условие истинно	Выполняется, пока условие ложно
Когда прекращается	Когда условие становится ложным	Когда условие становится истинным
Смысл	«Пока (условие верно) — делай»	«До тех пор, пока (условие не станет верным) — делай»

Сравнение на примере:

```
# while: выполняется, пока x < 5
x=1
while [ $x -lt 5 ]; do
    echo "while: x=$x"
```

```

    ((x++))
done
# Вывод: x=1, x=2, x=3, x=4

# until: выполняется, пока x не станет >= 5
x=1
until [ $x -ge 5 ]; do
    echo "until: x=$x"
    ((x++))
done
# Вывод: x=1, x=2, x=3, x=4

```

Ключевое различие:

- while продолжает цикл, пока условие истинно
- until продолжает цикл, пока условие ложно (то есть до тех пор, пока условие не станет истинным)

Оба цикла могут давать одинаковый результат при использовании противоположных условий:

```

while [ $x -lt 5 ]    # эквивалентно
until [ $x -ge 5 ]    # потому что "меньше 5" = "не больше или равно 5"

```